

```
In [ ]: ###
# CHAPTER 2 CODE EXAMPLES
###
from pyspark.sql import SparkSession
```

```
In [11]: spark = SparkSession.builder.appName("Chapter2Examples").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")
# Reading data is a lazy transformation. Spark peeks at a few rows only to infer the schema
# but waits to perform the operation until the take or show action
flightData2015 = spark \
    .read \
    .option("inferSchema", "true") \
    .option("header", "true") \
    .csv("flight-data/2015-summary.csv")

flightData2015.take(5)
```

```
Out[11]: [Row(DEST_COUNTRY_NAME='United States', ORIGIN_COUNTRY_NAME='Romania', count=15),
Row(DEST_COUNTRY_NAME='United States', ORIGIN_COUNTRY_NAME='Croatia', count=1),
Row(DEST_COUNTRY_NAME='United States', ORIGIN_COUNTRY_NAME='Ireland', count=344),
Row(DEST_COUNTRY_NAME='Egypt', ORIGIN_COUNTRY_NAME='United States', count=15),
Row(DEST_COUNTRY_NAME='United States', ORIGIN_COUNTRY_NAME='India', count=62)]
```



Figure 2-7. Reading a CSV file into a DataFrame and converting it to a local array or list of rows

```
In [12]: # sort data according to the count column
count = flightData2015.sort("count")
count.explain() # see the generated plan before the transformation is executed
count.take(3)

# Read plan top to bottom - top being end result and bottom being source of data
# Sort is a wide transformation as rows need to be compared across partitions

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Sort [count#98 ASC NULLS FIRST], true, 0
   +- Exchange rangepartitioning(count#98 ASC NULLS FIRST, 5), ENSURE_REQUIREMENTS, [plan_id=262]
      +- FileScan csv [DEST_COUNTRY_NAME#96,ORIGIN_COUNTRY_NAME#97,count#98] Batched: false, DataFilters: [], Forma
t: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/xavierruiz/Documents/repositories/x-ruiz/playground-data-
e..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<DEST_COUNTRY_NAME:string,ORIGIN_COUNTRY_NAME:stri
ng,count:int>
```

```
Out[12]: [Row(DEST_COUNTRY_NAME='Moldova', ORIGIN_COUNTRY_NAME='United States', count=1),
Row(DEST_COUNTRY_NAME='United States', ORIGIN_COUNTRY_NAME='Croatia', count=1),
Row(DEST_COUNTRY_NAME='United States', ORIGIN_COUNTRY_NAME='Singapore', count=1)]
```

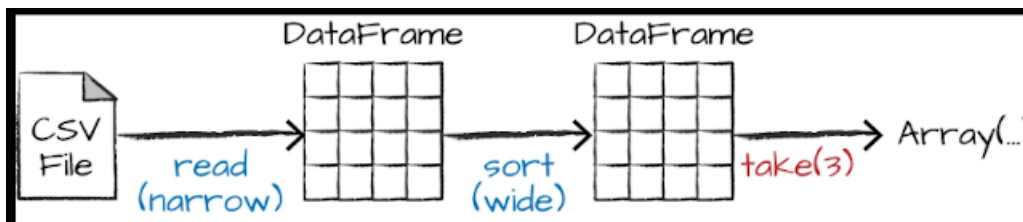
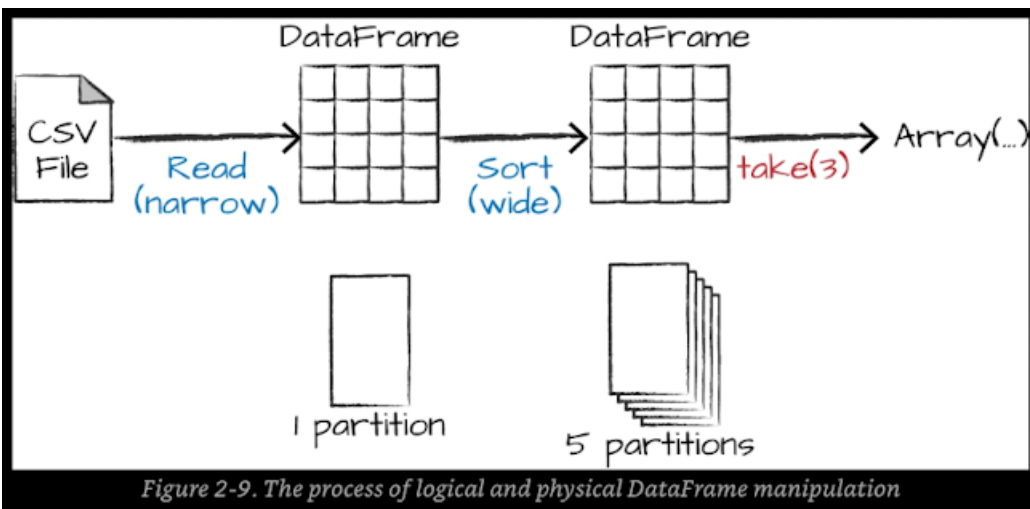


Figure 2-8. Reading, sorting, and collecting a DataFrame

```
In [4]: # Reduce the # of output partitions to 5, by default it is set to 200
spark.conf.set("spark.sql.shuffle.partitions", "5")
flightData2015.sort("count").take(2) # action that triggers the transformation
```

```
Out[4]: [Row(DEST_COUNTRY_NAME='United States', ORIGIN_COUNTRY_NAME='Singapore', count=1),
Row(DEST_COUNTRY_NAME='Moldova', ORIGIN_COUNTRY_NAME='United States', count=1)]
```



```
In [5]: # Turn any Dataframe into a table or a view to query with SQL
# there is no performance difference as both "compile" to the
# same underlying plan.
flightData2015.createOrReplaceTempView("flight_data_2015")

# Compare plans of the sql method vs dataframe method
sqlWay = spark.sql("""
SELECT DEST_COUNTRY_NAME, count(1)
FROM flight_data_2015
GROUP BY DEST_COUNTRY_NAME
""")

dataFrameWay = flightData2015 \
    .groupBy("DEST_COUNTRY_NAME") \
    .count()

sqlWay.explain() # action to show plan
dataFrameWay.explain() # action to show plan

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[DEST_COUNTRY_NAME#17], functions=[count(1)])
   +- Exchange hashpartitioning(DEST_COUNTRY_NAME#17, 5), ENSURE_REQUIREMENTS, [plan_id=55]
      +- HashAggregate(keys=[DEST_COUNTRY_NAME#17], functions=[partial_count(1)])
         +- FileScan csv [DEST_COUNTRY_NAME#17] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths) [file:/Users/xavierruiz/Documents/repositories/x-ruiz/playground-data-e..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<DEST_COUNTRY_NAME:string>

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[DEST_COUNTRY_NAME#17], functions=[count(1)])
   +- Exchange hashpartitioning(DEST_COUNTRY_NAME#17, 5), ENSURE_REQUIREMENTS, [plan_id=68]
      +- HashAggregate(keys=[DEST_COUNTRY_NAME#17], functions=[partial_count(1)])
         +- FileScan csv [DEST_COUNTRY_NAME#17] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths) [file:/Users/xavierruiz/Documents/repositories/x-ruiz/playground-data-e..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<DEST_COUNTRY_NAME:string>
```

```
In [6]: from pyspark.sql.functions import max

# Transformation that filters down to one row (wide)
# This is a wide transformation because one partition cannot compute the max by itself
# without reading other partitions
flightData2015.select(max("count")).take(1)
```

```
Out[6]: [Row(max(count)=370002)]
```

```
In [7]: # Find out the top 5 destination countries
maxSql = spark.sql("""
SELECT DEST_COUNTRY_NAME, sum(count) as TRIP_COUNT
FROM flight_data_2015
GROUP BY DEST_COUNTRY_NAME
ORDER BY TRIP_COUNT DESC
LIMIT 5
""")
```

```
maxSql.explain()
maxSql.show()
```

== Physical Plan ==

AdaptiveSparkPlan isFinalPlan=false

```
+-- TakeOrderedAndProject(limit=5, orderBy=[TRIP_COUNT#45L DESC NULLS LAST], output=[DEST_COUNTRY_NAME#17,TRIP_COUNT#45L])
  +- HashAggregate(keys=[DEST_COUNTRY_NAME#17], functions=[sum(count#19)])
    +- Exchange hashpartitioning(DEST_COUNTRY_NAME#17, 5), ENSURE_REQUIREMENTS, [plan_id=117]
      +- HashAggregate(keys=[DEST_COUNTRY_NAME#17], functions=[partial_sum(count#19)])
        +- FileScan csv [DEST_COUNTRY_NAME#17,count#19] Batched: false, DataFilters: [], Format: CSV, Location:
InMemoryFileIndex(1 paths)[file:/Users/xavierruiz/Documents/repositories/x-ruiz/playground-data-e..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<DEST_COUNTRY_NAME:string,count:int>
```

```
+-----+-----+
|DEST_COUNTRY_NAME|TRIP_COUNT|
+-----+-----+
|      United States|      411352|
|           Canada|       8399|
|           Mexico|       7140|
|   United Kingdom|       2025|
|           Japan|       1548|
+-----+-----+
```

In [8]: *# Dataframe version of the same thing*
`from pyspark.sql.functions import desc`

```
dataFrameWay = flightData2015 \
    .groupBy("DEST_COUNTRY_NAME") \
    .sum("count") \
    .withColumnRenamed("sum(count)", "TRIP_COUNT") \
    .sort(desc("TRIP_COUNT")) \
    .limit(5)
dataFrameWay.explain()
```

== Physical Plan ==

AdaptiveSparkPlan isFinalPlan=false

```
+-- TakeOrderedAndProject(limit=5, orderBy=[TRIP_COUNT#65L DESC NULLS LAST], output=[DEST_COUNTRY_NAME#17,TRIP_COUNT#65L])
  +- HashAggregate(keys=[DEST_COUNTRY_NAME#17], functions=[sum(count#19)])
    +- Exchange hashpartitioning(DEST_COUNTRY_NAME#17, 5), ENSURE_REQUIREMENTS, [plan_id=180]
      +- HashAggregate(keys=[DEST_COUNTRY_NAME#17], functions=[partial_sum(count#19)])
        +- FileScan csv [DEST_COUNTRY_NAME#17,count#19] Batched: false, DataFilters: [], Format: CSV, Location:
InMemoryFileIndex(1 paths)[file:/Users/xavierruiz/Documents/repositories/x-ruiz/playground-data-e..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<DEST_COUNTRY_NAME:string,count:int>
```

In [9]: `dataFrameWay.show()`

```
+-----+-----+
|DEST_COUNTRY_NAME|TRIP_COUNT|
+-----+-----+
|      United States|      411352|
|           Canada|       8399|
|           Mexico|       7140|
|   United Kingdom|       2025|
|           Japan|       1548|
+-----+-----+
```

